AI-TECH

Przykład 1 DO LABORATORIUM 2

z UCZENIA MASZYNOWEGO

Klasyfikacja nadzorowana

Szymon Zaporowski

Klasyfikacja z użyciem Maszyny Wektorów Nośnych (SVM) - przykłady

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Przykład 1

Maszyna Wektorów Nośnych (SVM) jest wszechstronnym modelem uczenia maszynowego za pomocą którego można dokonywać klasyfikacji zaróno liniowej jak i nieliniowej, regresji oraz detekcji elementów odstających. Główne zastosowanie SVM to klasyfikacja złożonych lecz niewielkich zbiorów danych.

Dokładny sposób działania SVM został przedstawiony w ramach wykładów z przedmiotu Uczenie maszynowe:

> wykład nr 7: https://enauczanie.pg.edu.pl/moodle/pluginfile.php/1174537/mod_resource/content/2/uczenie_maszynowe_7_rozszerzone.pdf

> wykład nr 8: https://enauczanie.pg.edu.pl/moodle/pluginfile.php/1184214/mod_resource/content/1/uczenie_maszynowe_8_rozszerzone.pdf

Oraz w materiałach pomocnicznych do wykładów:

> Wprowadzenie do SVM - wektory nośne, wyznaczanie marginesów:

https://colab.research.google.com/drive/1usd-0v8DUUx70TvyZGzFe66ZiC3MYxR2?usp=sharing

> Sztywne marginesy w SVM, prosta implementacja z użyciem Scikit_learn:
> https://colab.research.google.com/drive/1iq5UPWD1NLgL62SUUlyVq6sl3HvXw9QE?usp=sharing

> Miękki margines oraz różne funkcje jądra SVM: https://colab.research.google.com/drive/1g64Y1b6WCWS-GmK6yq5D1hraiHHKQmlm?usp=sharing

> Porównanie regresji logistycznej z SVM, podejście mutliklasowe do klasyfikacji z użyciem SVM: https://colab.research.google.com/drive/1EIIrzzg8i5RF8ZySi0ZKqWQDUP_u1bmE?usp=sharing

W tym przykładzie skupimy się na zastosowaniu SVM do klasyfikacji. W tym celu wykorzystamy znany z wykładów zbiór danych dotyczący diagnostyki nowotworów piersi

Najpierw dokonujemy załadowania bibliotek niezbędnych do uruchomienia ćwiczenia.

In [6]:
```
#Import bilbioteki scikit-learn i modułu dataset zawierającego zbiory danych
from sklearn import datasets
# Zaimporotwanie funkcji train_test_split do podziału zbioru na dane trenignowe i testowe
from sklearn.model_selection import train_test_split
# Zaimporotowanie modelu SVM z packietu sklearn
from sklearn import svm
#Import modułu metrics z pakietu scikit-learn do policzenia miary dokładności klasyfikatora
from sklearn import metrics
# Import modułów potrzebnych do tworzenia grafiki i obliczeń
import numpy as np #import biblioteki numpy do obliczeń
import pandas as pd # import biblioteki pandas do przetwarzania danych
import matplotlib.pyplot as plt #import biblioteki matplotlib i modułu pyplot
from matplotlib.colors import ListedColormap
import seaborn as sns #import biblioteki seaborn do tworzenia wykresów
from mpl_toolkits.mplot3d import Axes3D # import biblioteki potrzebnej do wizualizacji 3D
from sklearn.decomposition import PCA # import biblioteki niezbędnej do przeprowadzenia redukcji wymiarowości z użyciem PCA
%matplotlib inline
from sklearn.pipeline import Pipeline
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from mlxtend.plotting import plot_decision_regions
from sklearn.svm import SVC
```

Dokonujemy ładowania danych, wyświetlamy nazwy cech oraz etykiet

In [7]:
```
#Załadowanie zbioru danych
dataset = datasets.load_breast_cancer() #datasets.load_iris() #datasets.load_breast_cancer() #datasets.load_wine() #load_breast_cancer()

# Wypisanie zbioru nazw cech ze zbioru danych ( w tym przypadku 13 cech)
print("Features: ", dataset.feature_names)
```

```
#Wypisanie etykiet zawierających typy raka: "złośliwy", "łagodny"
print("Labels: ", dataset.target_names)
```

```
Features:  ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
Labels:  ['malignant' 'benign']
```

Sprawdzamy jak wyglądają wartości numeryczne danych poprzez wypisanie kształtu wektora danych oraz kilku pierwszych wartośći oraz wartości etykiet.

In [8]:
```
# Wypisanie kształtu danych (cech) ze zbioru
print("Kształt wektora danych",dataset.data.shape)
# Wypisanie wartości dla 5 pierwych wpisów w danych
print("Wartości pierwszych 5 rekordów:",dataset.data[0:1])
# Wypisanie etykiet oznaczających nowotwór (0:złośliwy, 1:łagodny)
print("Etykiety danych:",dataset.target)
```

```
Kształt wektora danych (569, 30)
Wartości pierwszych 5 rekordów: [[1.799e+01 1.038e+01 1.228e+02 1.001e+03 1.184e-01 2.776e-01 3.001e-01
  1.471e-01 2.419e-01 7.871e-02 1.095e+00 9.053e-01 8.589e+00 1.534e+02
  6.399e-03 4.904e-02 5.373e-02 1.587e-02 3.003e-02 6.193e-03 2.538e+01
  1.733e+01 1.846e+02 2.019e+03 1.622e-01 6.656e-01 7.119e-01 2.654e-01
  4.601e-01 1.189e-01]]
Etykiety danych: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

Następnie dokonywany jest podział zbioru na dane testow i treningowe z wykorzystaniem polecenia

```
# X_train, X_test, y_train, y_test = train_test_split(dataset.data, dataset.target, test_size=0.30,random_state=109)
```

Należy zwrócić uwagę na parametr

```
# test_size=0.30
```

Poprzez który ustalana jest wielkość zbioru testowego oraz treningowego, wartość 0.30 oznacza 30% zbioru testowego i 70% zbioru treningowego.

Dla sprawdzenia czy podział został dokonany poprawnie wyświetlamy wartości dla wszystkich podzielonych zbiorów.

In [9]:
```
# Podział zbioru na dane treningowe i testowe
X_train, X_test, y_train, y_test = train_test_split(dataset.data, dataset.target, test_size=0.10,random_state=109) # Rozmiar zbiorów to 70%
print("zbiór trenignowy:", X_train)
print("zbiór trenignowy - etykiety:", y_train)
print("zbiór testowy:", X_test)
print("zbiór testowy - etykiety:", y_test)
```

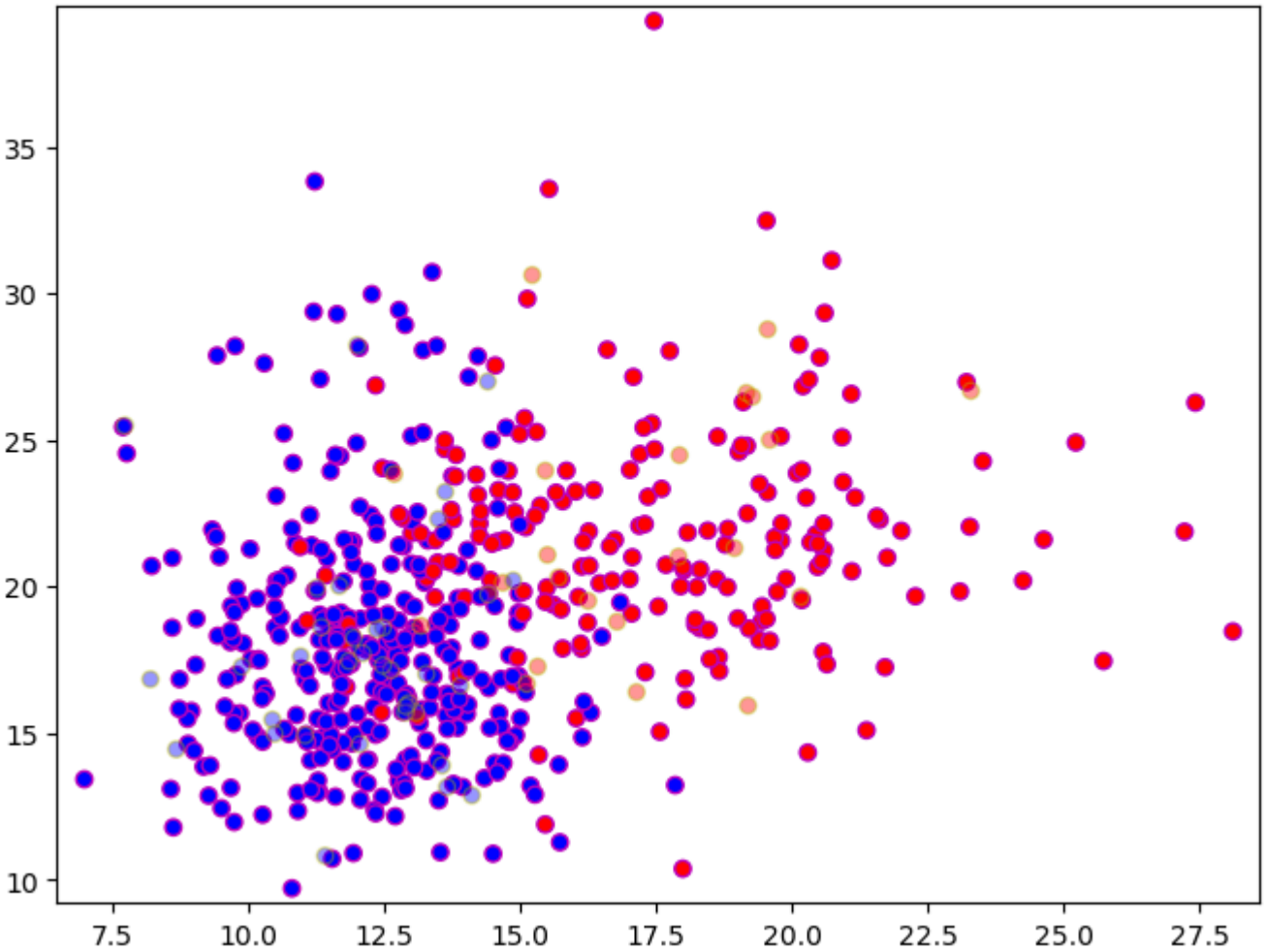
```
zbiór trenignowy: [[1.298e+01 1.935e+01 8.452e+01 ... 9.858e-02 3.596e-01 9.166e-02]
 [8.950e+00 1.576e+01 5.874e+01 ... 3.846e-02 1.652e-01 7.722e-02]
 [1.603e+01 1.551e+01 1.058e+02 ... 1.981e-01 3.019e-01 9.124e-02]
 ...
 [1.611e+01 1.805e+01 1.051e+02 ... 1.216e-01 2.792e-01 8.158e-02]
 [1.048e+01 1.986e+01 6.672e+01 ... 6.736e-02 2.883e-01 7.748e-02]
 [1.729e+01 2.213e+01 1.144e+02 ... 1.528e-01 3.067e-01 7.484e-02]]
zbiór trenignowy - etykiety: [1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0
 1 1 1 1 1 0 0 1 1 0 1 1 0 1 0 1 1 0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0
 0 0 0 0 1 1 0 1 1 1 0 1 1 0 0 1 0 0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0
 1 1 1 0 1 1 1 1 1 1 0 0 1 0 1 1 0 0 0 0 1 0 1 0 1 1 1 1 0 0 1 0 1 1 1 1 1
 0 1 0 1 0 1 0 1 0 0 0 0 1 1 0 1 0 1 0 0 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1
 1 1 0 1 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1
 1 0 0 0 1 1 1 0 1 1 1 0 0 1 1 1 0 1 0 0 0 1 0 1 1 1 0 1 0 1 0 0 1 1 1 1 0
 0 1 1 1 1 1 0 1 1 1 0 0 1 0 1 0 0 0 0 1 1 0 0 1 1 1 1 1 1 1 1 1 0 1 0 0 0
 1 1 0 1 0 1 1 1 0 1 1 0 0 1 0 1 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0
 0 0 1 0 0 1 1 1 1 1 0 0 0 1 1 1 0 1 0 1 1 1 1 1 0 0 0 1 1 1 0 1 0 0 0 0 0
 1 1 1 1 0 1 1 1 0 1 1 0 1 0 1 0 1 0 1 1 0 1 1 1 1 1 1 0 0 0 0 1 0 1 1 1 0
 1 1 1 0 1 0 0 1 1 1 0 1 0 1 0 0 0 0 1 1 0 0 1 0 1 1 1 0 1 0 1 1 0 1 1 0 0
 1 0 1 0 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 1 1 1 0 0 1 1 1 0 0 1 1
 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 0 0 0 1 1 0 1 0 1 0]
zbiór testowy: [[1.167e+01 2.002e+01 7.521e+01 ... 8.120e-02 3.206e-01 8.950e-02]
 [1.194e+01 1.824e+01 7.571e+01 ... 6.296e-02 2.785e-01 7.408e-02]
 [1.916e+01 2.660e+01 1.262e+02 ... 1.872e-01 3.258e-01 9.720e-02]
 ...
 [8.196e+00 1.684e+01 5.171e+01 ... 2.564e-02 3.105e-01 7.409e-02]
 [1.894e+01 2.131e+01 1.236e+02 ... 1.789e-01 2.551e-01 6.589e-02]
 [1.290e+01 1.592e+01 8.374e+01 ... 1.012e-01 3.549e-01 8.118e-02]]
zbiór testowy - etykiety: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1
 0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 1 0 1]
```

Rysujemy na płaszczyźnie zbiór danych, dane odróżniające się kolorem - lekko zanikające to dane testowe, pozostałe dane to dane treningowe.

In [10]:

```
X = dataset.data
y = dataset.target

x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5

plt.figure(2, figsize=(8, 6))

# Rysowanie danych treningowych
#plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Set1,
#            edgecolor='k')

# Rysowanie punktów należacych do zbioru treningowego
```

```
cm_bright = ListedColormap(['#FF0000', '#0000FF'])
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=cm_bright,
                   edgecolors='m')
        # Rysowanie punktów należacych do zbioru testowego
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=cm_bright,
                   edgecolors='y', alpha=0.4)


plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)

plt.show()
```



Patrząc na wykres reprzentacji klas należacych do zbioru danych przedstawionych na płaszczyźnie można łatwo zauważyć, iż idealna separacja klas z wykorzystaniem prostej linii separucjącej może być niemożliwa. Wykorzystamy klasyfikator oparty o SVM z jądrem liniowym. Klasyfikator został

zaimportowany z użyciem biblioteki scikit-learn. Poniżej pokazano komendę odpowiedzialną za import klasyfikatora. Jak widać istnieje kilka parametrów, które należy wpisać. Pierwszy z nich jest rodzaj jądra czyli kernel. Do wyboru jest kilka opcji, zaczynąc od najprosztszego czyli linowego ukrywającego się pod nazwą "linear", poprzez jądra wielomianowe "poly" czy czy też funkcję rbf czyli "rbf". Oprócz tego istnieją inne istotne parametry takie jak współczynnik C odpowiadający za określenie wektorów nośnych oraz lini separującej i marginesu.

```
clf = svm.SVC(kernel='linear', C=1 )
```

Pełna dokumentacja funkcji znajduje się na stronie:https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html?highlight=svc#sklearn.svm.SVC

Dodatkowo do sprawdzenia jakości modelu wykorzystamy miarę Accuracy, która powinna być państwu znana. Dla przypomnienia, jest to stosunek poprwnie dokonanych klasyfikacji do wszystkich dokonanych klasyfikacji.

Gdyby zapisać to za pomocą wzoru otrzymamy:

$$Accuracy = \frac{TruePositive + TrueNegative}{TruePositive + TrueNegative + FalsePositive + FalseNegative}$$

Oprócz tego wykorzystane zostaną dwie inne miary: Precision oraz Recall. Precision jest definowana jako stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie pozytywnych wyników i wyników fałszywie pozytywnych.

Zapisując to we wzorze otrzymamy:

$$Precision = \frac{TruePositive}{TruePositive + FalsePositive}$$

Podobnie w przypadku miary Recall, która jest stosunkiem wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozytywnych i fałszywie negatywnych.

Zapisując to za pomocą wzoru otrzymamy:

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative}$$

In [11]:
```
#Inicjalizacja klasyfikatora bazującego na SVM
clf = svm.SVC(kernel='rbf')#,probability=True) # kernel - opcje: rbf - domyślna, linear -linioye, poly - wielomianowe, dodatkowy parametr de
#Trening modelu z wykorzystaniem danych treningowych
clf.fit(X_train, y_train)
#Predykcja odpowiedzi modelu na dane testowe
y_pred = clf.predict(X_test)
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Miara accuracy:",metrics.accuracy_score(y_test, y_pred))
```

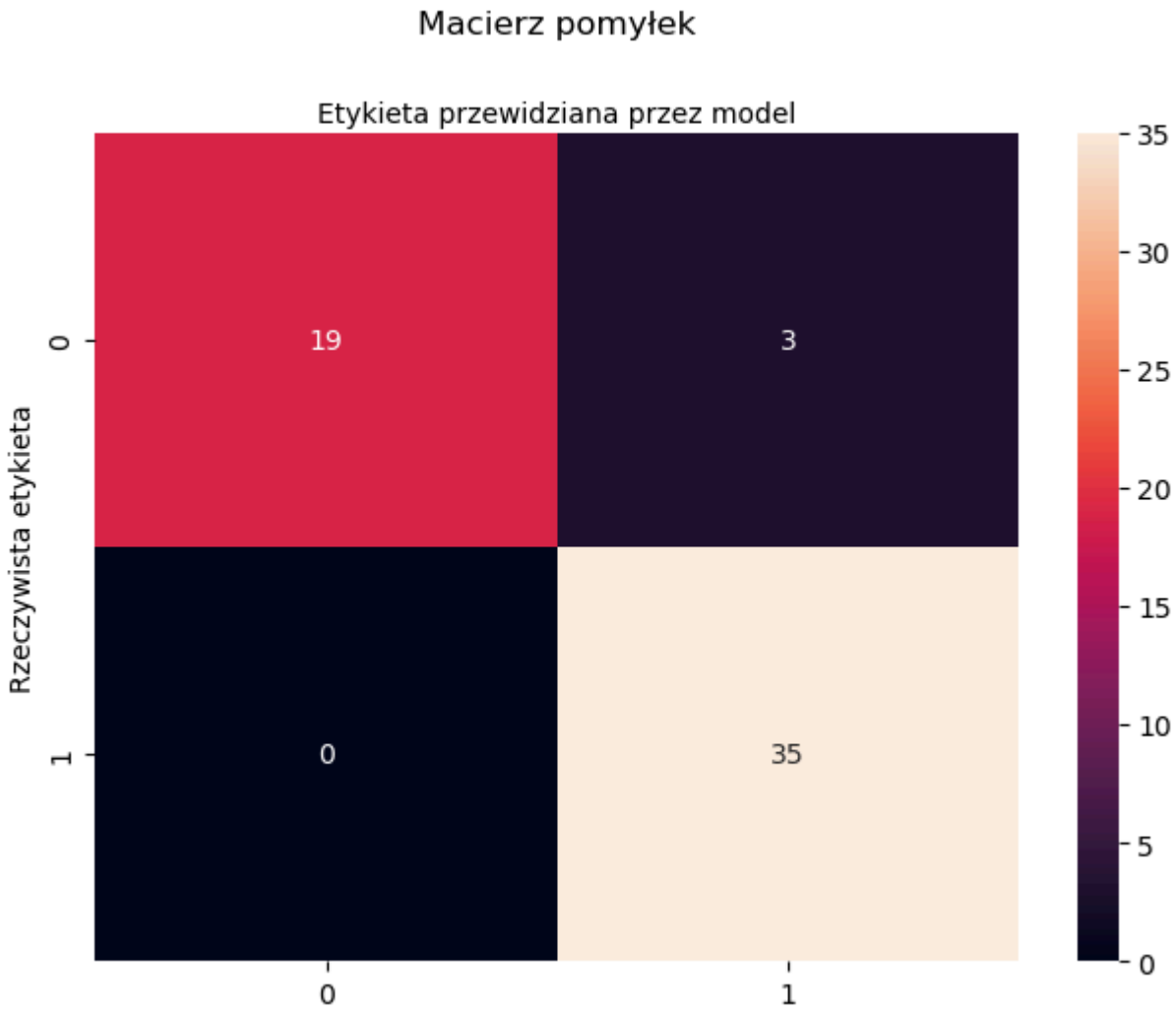
```
y_score = clf.decision_function(X_test)
# Obliczenia wartości miary precison dla modelu: (precision to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie p
print("Wartość miary precision dla wytrenowanego modelu:",metrics.precision_score(y_test, y_pred))
# Obliczenia wartości miary recall dla modelu: (recall to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozyty
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test, y_pred))
```

```
Miara accuracy: 0.9473684210526315
Wartość miary precision dla wytrenowanego modelu: 0.9210526315789473
Wartość miary recall dla wytrenowanego modelu: 1.0
```

Aby sprawdzić dokładnie jak wyglądała klasyfikacja i gdzie nastąpiły pomyłki wykorzystuje się macierz pomyłek. Poniżej pokazano kod pozwalający na graficzna interpretację wyników z wykorzystaniem macierzy pomyłek.

In [12]:
```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[12]:
```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```



Spróbujmy teraz wykorzystać inne jądro i sprawdzimy czy wpłynie to na wyniki klasyfikacji. Tym razem wykorzystane zostanie jądro Radial Baisis Function.

In [13]:
```
#Inicjalizacja klasyfikatora bazującego na SVM
clf = svm.SVC(kernel='rbf', C=100)#,probability=True) # kernel - opcje: rbf - domyślna, linear -linioye, poly - wielomianowe, dodatkowy par
#Trening modelu z wykorzystaniem danych treningowych
clf.fit(X_train, y_train)
#Predykcja odpowiedzi modelu na dane testowe
y_pred = clf.predict(X_test)
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Miara accuracy:",metrics.accuracy_score(y_test, y_pred))
y_score = clf.decision_function(X_test)
# Obliczenia wartości miary precison dla modelu: (precision to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie p
print("Wartość miary precision dla wytrenowanego modelu:",metrics.precision_score(y_test, y_pred))
```

```
# Obliczenia wartości miary recall dla modelu: (recall to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozyty
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test, y_pred))
```

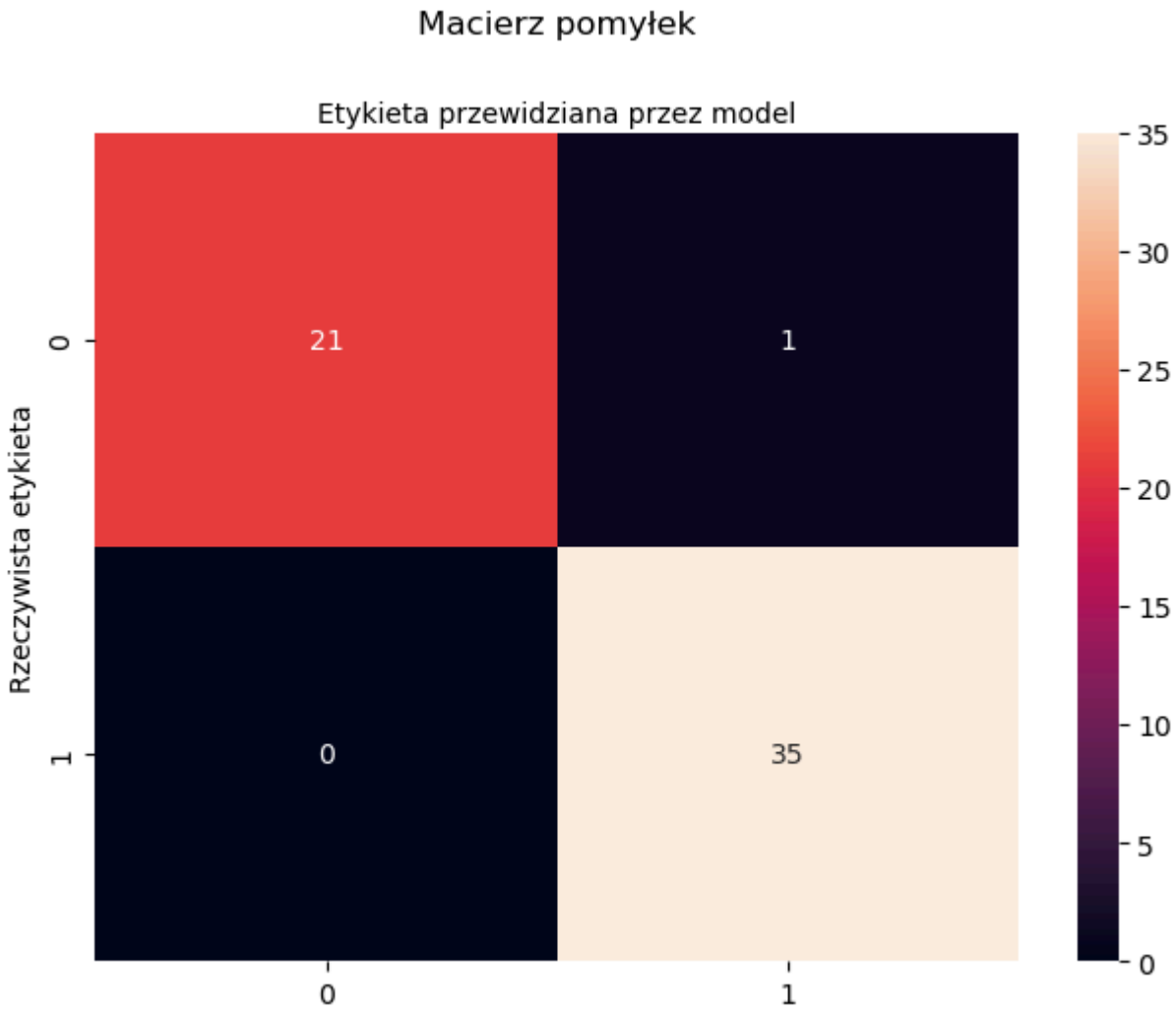
```
Miara accuracy: 0.9824561403508771
Wartość miary precision dla wytrenowanego modelu: 0.9722222222222222
Wartość miary recall dla wytrenowanego modelu: 1.0
```

In [14]:

```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[14]:

```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```



Udało się trochę poprawić wyniki klasyfikacji, natomiast jeszcze nie udało się uzyskać 100%.

Wykorzystamy teraz inną funkcję znajdująca sie w pakiecie scikit-learn do zaimplementowania klasyfikatora typu SVM. Użyte zostanie jądro liniowe i funkcja straty "hinge".

```
svm_clf = Pipeline ([
                     ("scaler", StandardScaler()),
                     ("linear_svc",LinearSVC(C=1, loss="hinge")),
])
```

Dodatkowo dokonano przekształcenia na danych - zostały one przeskalowane z użyciem funkcji StandardScaler. Funkcja ta standaryzuje dane, usuwając średnią i skalując je do wariancji jednostkowej

Funkcję oblicza się jako:

$$z = \frac{(x - u)}{s}$$

gdzie zmienna *u* jest rozumiana jako średnia wartość ze zbioru treningowego natomiast *s* jest to odchylenie standardowe wartości ze zbioru treningowego, natomiast *x* to oryginalna wartość ze zbioru treningowego.

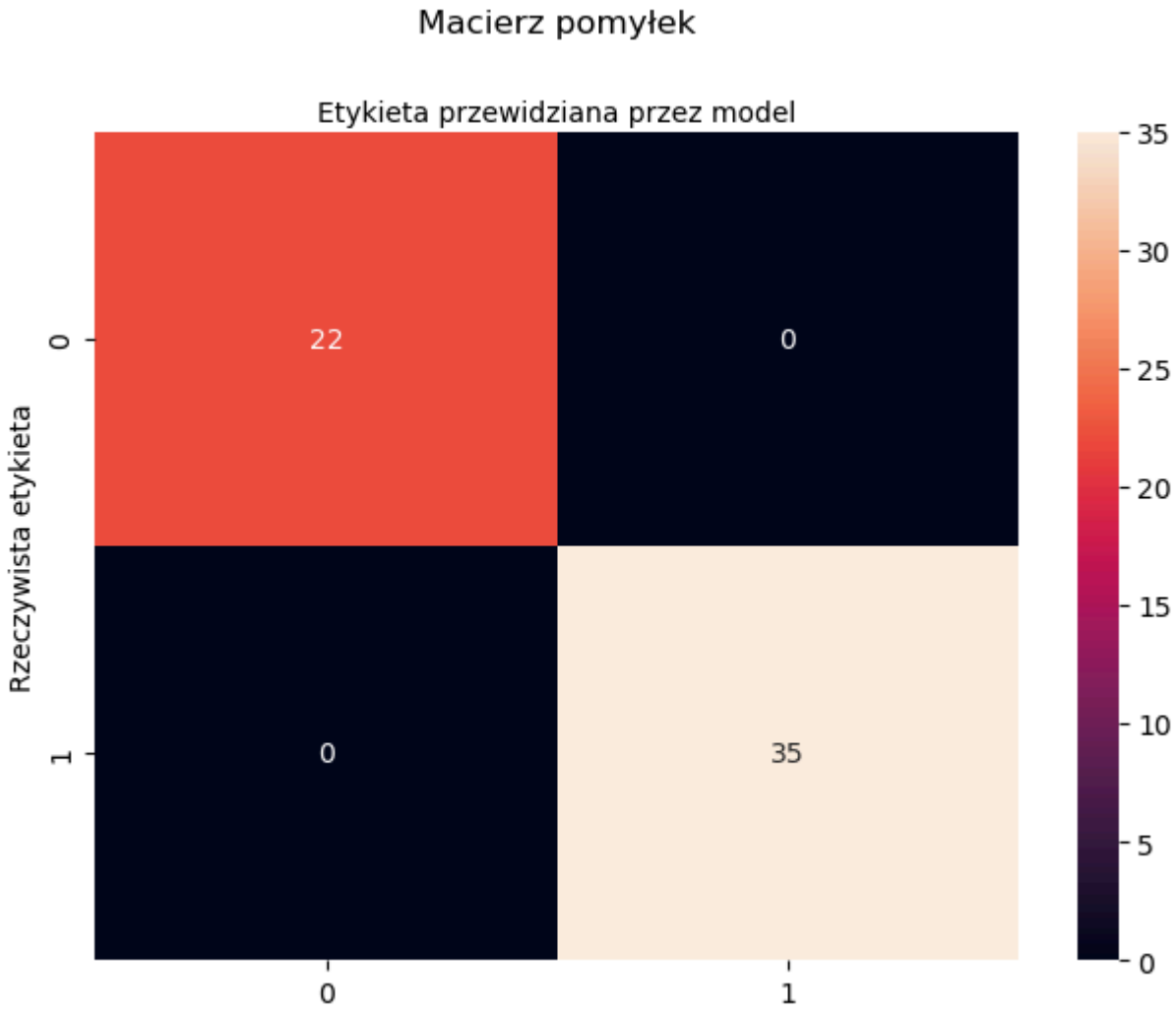
In [15]:
```
svm_clf = Pipeline ([
                     ("scaler", StandardScaler()),
                     ("linear_svc",LinearSVC(C=1, loss="hinge")),
])
svm_clf.fit(X_train,y_train)
y_pred_svm = svm_clf.predict(X_test)
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Miara accuracy:",metrics.accuracy_score(y_test, y_pred_svm))
y_score = svm_clf.decision_function(X_test)
# Obliczenia wartości miary precison dla modelu: (precision to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie p
print("Wartość miary precision dla wytrenowanego modelu:",metrics.precision_score(y_test,y_pred_svm))
# Obliczenia wartości miary recall dla modelu: (recall to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozyty
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test, y_pred_svm))
```

```
Miara accuracy: 1.0
Wartość miary precision dla wytrenowanego modelu: 1.0
Wartość miary recall dla wytrenowanego modelu: 1.0
```

```
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:1258: ConvergenceWarning: Liblinear failed to converge, increase the number
of iterations.
  warnings.warn(
```

In [16]:
```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred_svm)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[16]: Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')



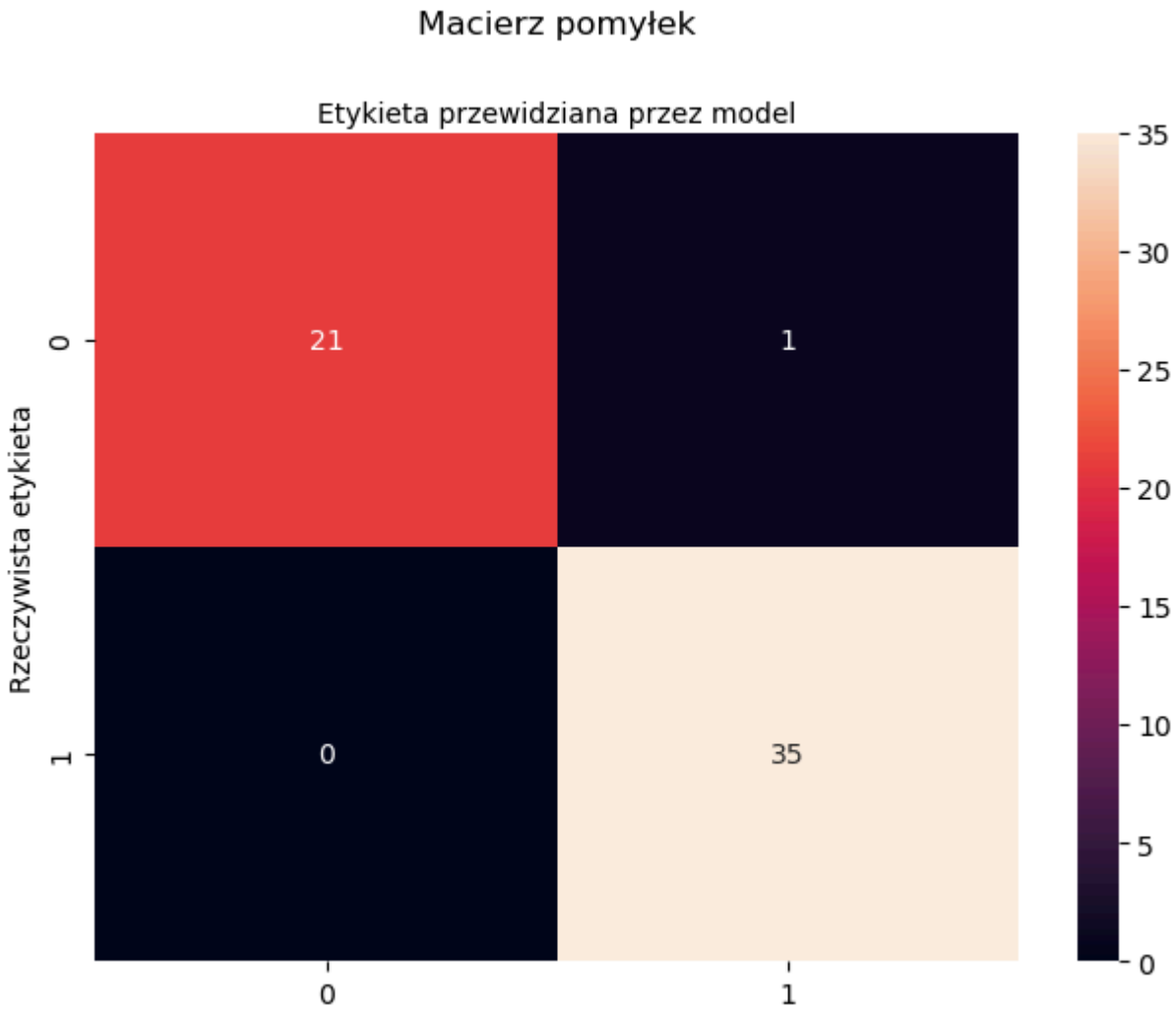
Jak widać pomimo zastosowania liniowego klasyfikatora SVM udało się uzyskać 100% skuteczności. Jest to zasługa zastosowania standaryzacji danych poprzez ich skalowanie. Wbrew pozorom skalowanie danych pozwala na poprawienie wyników klasyfikacji.

Uważny obserwator zauważy pewną nieścisłość. Dlaczego miara Accuracy na rysunku nie zgadza się z tą wyświetloną pod rysunkiem

In [17]:
```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
```

```
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[17]: Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')



In [19]:
```
#Inicjalizacja klasyfikatora bazującego na SVM
clf = svm.SVC(kernel='linear',  decision_function_shape='ovo')#,probability=True) # kernel - opcje: rbf - domyślna, linear -linioye, poly -
#Trening modelu z wykorzystaniem danych treningowych
```

```
clf.fit(X_train, y_train)
#Predykcja odpowiedzi modelu na dane testowe
y_pred = clf.predict(X_test)
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Dokładność:",metrics.accuracy_score(y_test, y_pred))
y_score = clf.decision_function(X_test)
# Obliczenia wartości miary precisson dla modelu: (precission to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie
print("Wartość miary precission dla wytrenowanego modelu:",metrics.precision_score(y_test, y_pred))
# Obliczenia wartości miary recall dla modelu: (recal to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozytyw
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test, y_pred))
```

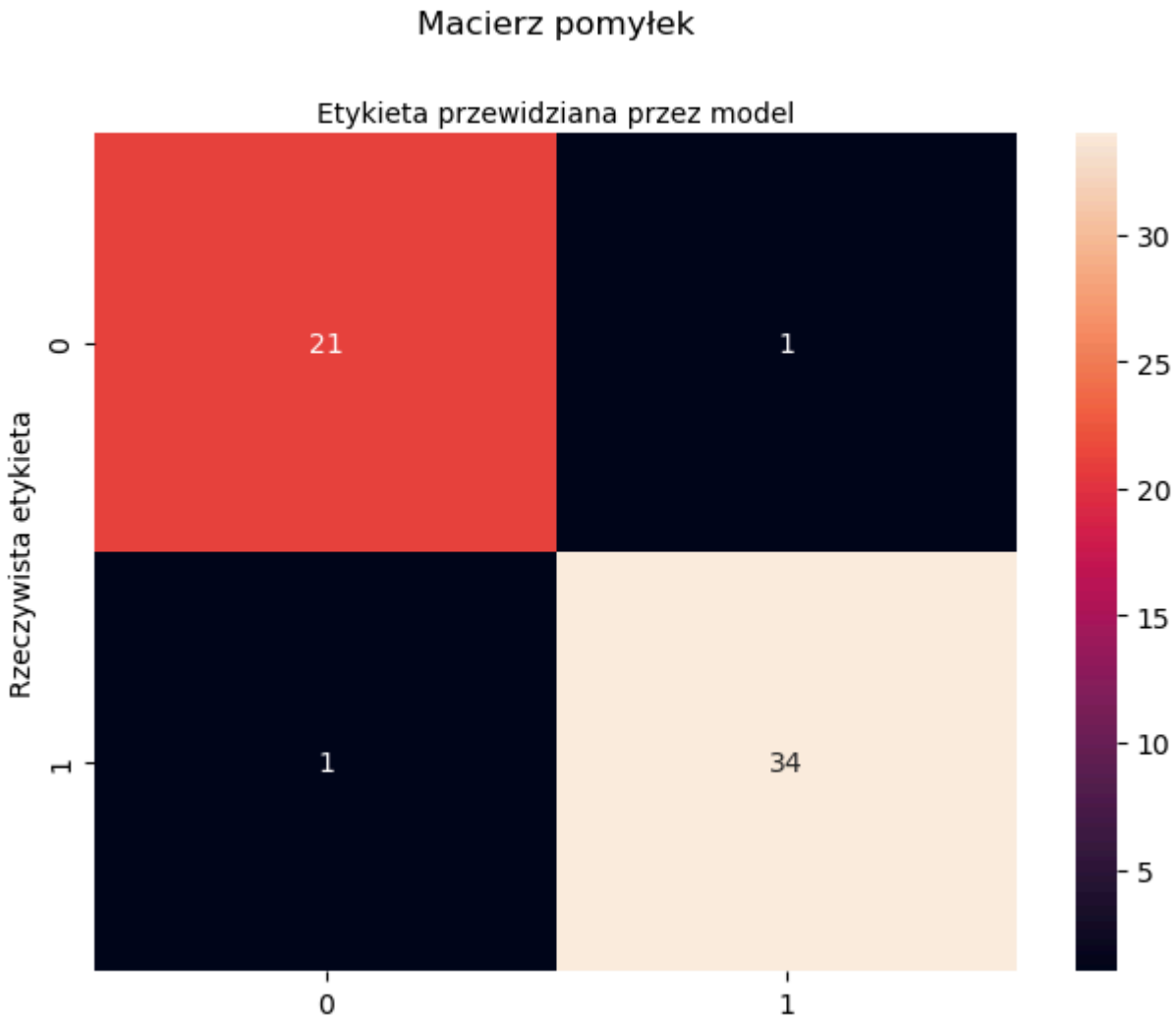
```
Dokładność: 0.9649122807017544
Wartość miary precission dla wytrenowanego modelu: 0.9714285714285714
Wartość miary recall dla wytrenowanego modelu: 0.9714285714285714
```

In [20]:

```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[20]:

```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```



In [21]:
```
#Inicjalizacja klasyfikatora bazującego na SVM
clf = svm.SVC(kernel='linear',  decision_function_shape='ovr')#,probability=True) # kernel - opcje: rbf - domyślna, linear -linioye, poly -
#Trening modelu z wykorzystaniem danych treningowych
clf.fit(X_train, y_train)
#Predykcja odpowiedzi modelu na dane testowe
y_pred = clf.predict(X_test)
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Dokładność:",metrics.accuracy_score(y_test, y_pred))
y_score = clf.decision_function(X_test)
# Obliczenia wartości miary precisson dla modelu: (precission to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie
print("Wartość miary precission dla wytrenowanego modelu:",metrics.precision_score(y_test, y_pred))
# Obliczenia wartości miary recall dla modelu: (recal to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozytyw
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test, y_pred))
```

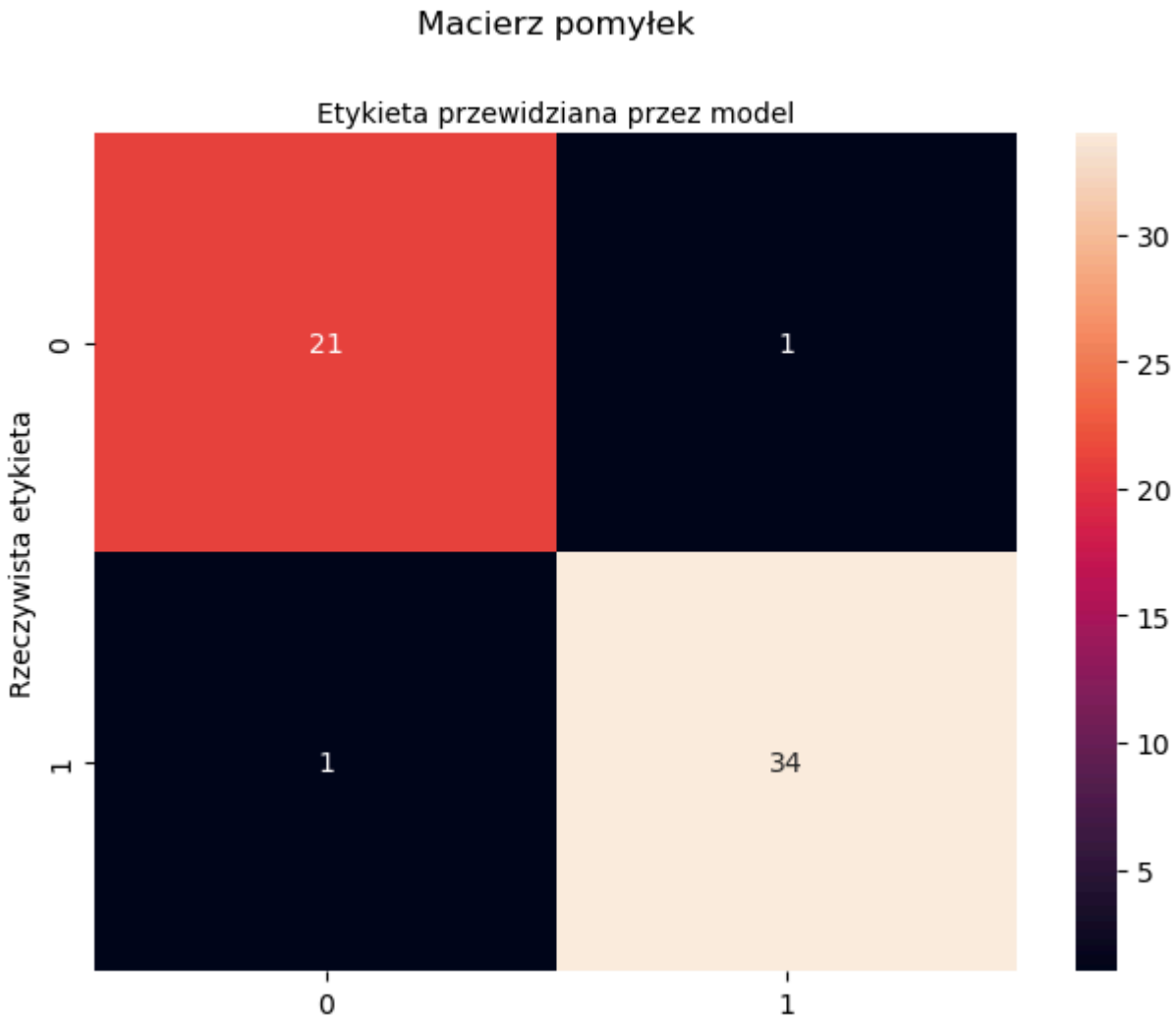
```
Dokładność: 0.9649122807017544
Wartość miary precission dla wytrenowanego modelu: 0.9714285714285714
Wartość miary recall dla wytrenowanego modelu: 0.9714285714285714
```

In [22]:

```python
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[22]:

```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```



Zadanie 1

Bazując na powyższym przykładzie sprawdzić jak będzie działał SVM implementowany przy użyciu `svm.SVC()` z innymi nastawami - zmienić rodzaj jądra oraz jego stopień i wskazać wpływ na metryki accuracy, preccison i recall. W przypadku jądra rbf wykorzystać współczynnik *gamma*. Wybrać najlepsze nastawy i wykorzystać je w Zadaniu 2.

Zadanie 2

Zmienić podział zbioru danych co 10% zaczynając od podziału 10% danych treningowych - 90% danych testowych. Czy widać znaczącą różnicę w wynikach metryk accuracy, preccison i recall? Dla jakich wartośći uzyskano optymalne wyniki? Czy zaobserwowano zjawisko underfittingu i overfittingu? Jeśli tak to w jaki sposób się ono objawiało?

Zadanie 3

Wykorzystując *Pipeline* i *StandardScaler()* przeprowadzić ponownie dobór hiperparametrów dla różnych funkcji jądra. Porównać otrzymane wyniki z wynikami z zadania pierwszego.

In [ ]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score

print("ZADANIE 1")
print("Różne jądra SVM:\n")
for kernel in ['linear', 'poly', 'rbf', 'sigmoid']:
    if kernel == 'poly':
        clf = svm.SVC(kernel=kernel, degree=3)
    elif kernel == 'rbf':
        clf = svm.SVC(kernel=kernel, gamma='auto')
    else:
        clf = svm.SVC(kernel=kernel)

    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(f"Kernel: {kernel}")
    print(f"\tAccuracy: {accuracy_score(y_test, y_pred):.4f}")
    print(f"\tPrecision: {precision_score(y_test, y_pred):.4f}")
    print(f"\tRecall: {recall_score(y_test, y_pred):.4f}\n")
```

```
ZADANIE 1
Różne jądra SVM:

Kernel: linear
        Accuracy: 0.9649
        Precision: 0.9714
        Recall: 0.9714

Kernel: poly
        Accuracy: 0.9298
        Precision: 0.8974
        Recall: 1.0000

Kernel: rbf
        Accuracy: 0.6140
        Precision: 0.6140
        Recall: 1.0000

Kernel: sigmoid
        Accuracy: 0.3333
        Precision: 0.4595
        Recall: 0.4857
```

Komentarz / wnioski do zadania 1:

Najlepsze wyniki dla tych danych osiągnął klasyfikator z jądrem:

- liniowym (linear) -> Accuracy: 0.9649
- wielomianowym (poly) -> Accuracy: 0.9298

Zbiór danych breast cancer wydaje się być dobrze separowalny liniowo, stąd wysoka skuteczność w przypadku jądra liniowego na poziomie ponad 96%.

Jądro wielomianowe dało nieco gorszy wynik ogólny (~93%).

Natomiast jądra rbf (Radial Basis Function) i sigmoid bez wcześniejszego skalowania danych poradziły sobie znacząco gorzej:

- rbf: Accuracy: 0.6140
- sigmoid: Accuracy: 0.3333

Prawdopodobnie jest to spowodowane tym, że cechy w zbiorze mają mocno różniące się zakresy wartości. A to mocno negatywnie wpływa na modele opierające się na miarach odległości.

In [26]:
```
print("ZADANIE 2)")
print("Proporcje podziału danych na zbiory treningowy i testowy\n")

for test_sz in np.arange(0.9, 0.0, -0.1):
    X_tr, X_te, y_tr, y_te = train_test_split(dataset.data, dataset.target, test_size=test_sz, random_state=109)
    clf = svm.SVC(kernel='linear')
    clf.fit(X_tr, y_tr)
    y_pred_te = clf.predict(X_te)
    y_pred_tr = clf.predict(X_tr)

    acc_te = accuracy_score(y_te, y_pred_te)
    acc_tr = accuracy_score(y_tr, y_pred_tr)

    print(f"Podział testowy: {test_sz:.1f} (Treningowy: {1-test_sz:.1f}) | Acc Train: {acc_tr:.4f} | Acc Test: {acc_te:.4f}")
```

```
ZADANIE 2)
Proporcje podziału danych na zbiory treningowy i testowy

Podział testowy: 0.9 (Treningowy: 0.1) | Acc Train: 1.0000 | Acc Test: 0.9181
Podział testowy: 0.8 (Treningowy: 0.2) | Acc Train: 0.9646 | Acc Test: 0.9452
Podział testowy: 0.7 (Treningowy: 0.3) | Acc Train: 0.9706 | Acc Test: 0.9449
Podział testowy: 0.6 (Treningowy: 0.4) | Acc Train: 0.9604 | Acc Test: 0.9591
Podział testowy: 0.5 (Treningowy: 0.5) | Acc Train: 0.9648 | Acc Test: 0.9649
Podział testowy: 0.4 (Treningowy: 0.6) | Acc Train: 0.9560 | Acc Test: 0.9649
Podział testowy: 0.3 (Treningowy: 0.7) | Acc Train: 0.9623 | Acc Test: 0.9649
Podział testowy: 0.2 (Treningowy: 0.8) | Acc Train: 0.9626 | Acc Test: 0.9561
Podział testowy: 0.1 (Treningowy: 0.9) | Acc Train: 0.9629 | Acc Test: 0.9649
```

Wnioski do zadania 2:

Dla podziału z bardzo małą ilością danych treningowych (np. 10%, 20% na train i pozostałe 80%, 90% na test) wyraźnie widać zjawisko overfittingu (przeuczenia). Model wydaje się mocny patrząc na dokładność na zbiorze treningowym (nawet 100% dokładności), ale brakuje mu informacji, żeby poprawnie przewidywać nowe przykłady - dokładność na zbiorze testowym spada do ~92%, co już jest sporą różnicą w porównaniu do wyniku na zbiorze treningowym.

Od punktu około 30-40% danych testowych, metryki stabilizują się z dobrym wynikiem rzędu powtarzalnego ~96% dla train i test. Podział 70% train / 30% lub 80% train / 20% test wydaje się najbardziej stabilnym podziałem.

Wszystkie wyniki są ponad 90%, więc niedouczenia, underfittingu nie zauważyłem.

In [30]:
```
from sklearn.pipeline import make_pipeline

print("ZADANIE 3")
print("Pipeline i StandardScaler\n")
```

```python
for kernel in ['linear', 'poly', 'rbf', 'sigmoid']:
    if kernel == 'poly':
        clf = make_pipeline(StandardScaler(), svm.SVC(kernel=kernel, degree=3))
    elif kernel == 'rbf':
        clf = make_pipeline(StandardScaler(), svm.SVC(kernel=kernel, gamma='auto'))
    else:
        clf = make_pipeline(StandardScaler(), svm.SVC(kernel=kernel))

    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(f"Jądro: {kernel} + StandardScaler")
    print(f" Accuracy: {accuracy_score(y_test, y_pred):.4f}")
    print(f" Precision: {precision_score(y_test, y_pred):.4f}")
    print(f" Recall: {recall_score(y_test, y_pred):.4f}\n")
```

```
ZADANIE 3
Pipeline i StandardScaler

Jądro: linear + StandardScaler
 Accuracy: 1.0000
 Precision: 1.0000
 Recall: 1.0000

Jądro: poly + StandardScaler
 Accuracy: 0.9298
 Precision: 0.8974
 Recall: 1.0000

Jądro: rbf + StandardScaler
 Accuracy: 0.9825
 Precision: 0.9722
 Recall: 1.0000

Jądro: sigmoid + StandardScaler
 Accuracy: 1.0000
 Precision: 1.0000
 Recall: 1.0000
```

Wnioski do zadania 3:

Zastosowanie StandardScaler przyniosło bardzo zauważalną poprawę wyników dla karneli rbf i sigmoid, które wcześniej miały słabe wyniki. Dokładność rbf wzrosła z ~61% do 98%, a dla sigmoid z ~33% do 100%. Różnica pokazuje jasno, że SVM z nieliniowymi jądrami jest algorytmem bardzo wrażliwym na pierwotną skalę zmiennych decyzyjnych. Kiedy jedna cecha przyjmuje duże wartości duże, dominuje ona w obliczeniach odległości kosztem mikroparametrów. Skalowanie powoduje powrót stabilnej separacji płaszczyzn.